

Empowering Character-level Text Infilling by Eliminating Sub-Tokens

Houxing Ren^{1,3} Mingjie Zhan^{2*} Zhongyuan Wu² Hongsheng Li^{3,4,5*}

¹Shanghai Jiao Tong University ²SenseTime Research

³Shanghai Artificial Intelligence Laboratory ⁴CUHK MMLab ⁵CPII under InnoHK

{renhouxing,georgewzy01}@gmail.com

zhanmingjie@sensetime.com hsli@ee.cuhk.edu.hk

<https://raccoon.sensetime.com/code>

Abstract

In infilling tasks, sub-tokens, representing instances where a complete token is segmented into two parts, often emerge at the boundaries of prefixes, middles, and suffixes. Traditional methods focused on training models at the token level, leading to sub-optimal performance in character-level infilling tasks during the inference stage. Alternately, some approaches considered character-level infilling, but they relied on predicting sub-tokens in inference, yet this strategy diminished ability in character-level infilling tasks due to the large perplexity of the model on sub-tokens. In this paper, we introduce FIM-SE, which stands for Fill-In-the-Middle with both Starting and Ending character constraints. The proposed method addresses character-level infilling tasks by utilizing a line-level format to avoid predicting any sub-token in inference. In addition, we incorporate two special tokens to signify the rest of the incomplete lines, thereby enhancing generation guidance. Extensive experiments demonstrate that our proposed approach surpasses previous methods, offering a significant advantage. Code is available at <https://github.com/SenseLLM/FIM-SE>.

1 Introduction

The Transformer (Vaswani et al., 2017) decoder-only architecture has proven highly effective in various natural language processing (NLP) tasks. This success has paved the way for the development of advanced causal decoder-only models like GPT-4 (OpenAI, 2023), PaLM (Chowdhery et al., 2023; Anil et al., 2023b), Llama (Touvron et al., 2023a,b; Rozière et al., 2023), and Falcon (Penedo et al., 2023). These innovative models excel at generating coherent and contextually relevant responses to natural language prompts, showcasing state-of-the-art performance across various tasks, including question answering (Lewis et al., 2020b),

Table 1: Examples for random splitting with Llama tokenizer, where the red, blue, and green text indicates the prefix, the middle, and the suffix, respectively. These four rows represent the pieces after randomly splitting, the sentence after exchanging suffix and middle, tokenized results, and token IDs, respectively.

(a) The first splitting case.	
Pieces	A fine day.
Reorder	A f day. ine
Tokens	['A', '_f', 'day', '.', 'ine', '_']
IDs	[29909, 285, 3250, 29889, 457, 29871]
(b) The second splitting case.	
Pieces	A fine day.
Reorder	A fi day. ne
Tokens	['A', '_fi', 'day', '.', 'ne', '_']
IDs	[29909, 5713, 3250, 29889, 484, 29871]

logical reasoning (Kojima et al., 2022), and code synthesis (Li et al., 2023; Rozière et al., 2023).

Despite the success, the proficiency of these models is somewhat limited in tasks involving text infilling, which aims to generate text at a specific location within a prompt, while conditioning on both a prefix and a suffix. The main reason is their intrinsic left-to-right autoregressive design. To address this issue, CM3 (Aghajanyan et al., 2022) introduced the causal masking objective, placing a mask token at the intended fill location and completing the fill at the end. In contrast, FIM (Bavarian et al., 2022) proposed a fill-in-the-middle technique, which randomly divides documents into three segments and tags them with three special tokens. This technique then rearranges the middle and suffix segments, to use the prefix and the suffix to predict the middle segment in auto-regressive format. With these methods, decoder-only based models can effectively handle various infilling tasks

*Corresponding author.

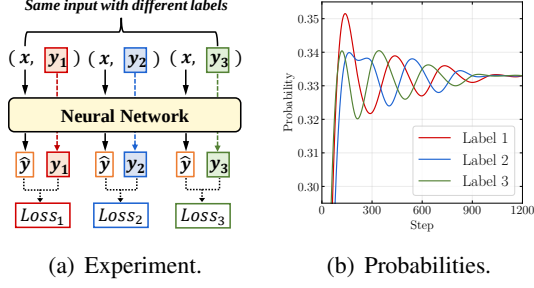


Figure 1: The probabilities of prediction when inconsistent labels appear in the training data.

and achieve excellent performance.

However, employing the aforementioned methods may introduce inconsistencies during training. This arises from the potential division of a single token into multiple sub-tokens, as exemplified in Table 1. As we can see, due to character-level random splitting, the same prefixes ([29909]) have inconsistent objectives (285 and 5713, respectively) in different cases. The inconsistent objectives will significantly impact the model’s perplexity, especially on sub-tokens. To illustrate, we construct a simple experiment on a classification task shown in Figure 1(a). The training data only contains three samples and they have the same input but different labels. We train a simple network on the training data and record the predicted probabilities of the three labels at each training step. As shown in Figure 1(b), the predicted probabilities for these three classes converge to 0.33, indicating a large perplexity of the model on the inconsistent objectives. The large perplexity on sub-tokens makes the probability of error increase when predicting a sub-token. This phenomenon is notably detrimental in sensitive tasks such as code completion, where even a minor error in any token can result in program malfunction. As a result, previous approaches have yet to fully inspire the potential of Transformer decoder-only models in infilling tasks.

To effectively address the issue, it is crucial to acknowledge and resolve an inherent conflict. (1) We need to avoid the model predicting sub-tokens. In the infilling training mode, the model’s perplexity in sub-tokens is large, resulting in the low accuracy of predicting sub-tokens. (2) We need to output a sub-token when the user only writes part of a token. Because it is necessary to ensure that the output fits the context. If we directly drop several tokens to make sure no sub-tokens exist, the model’s output may no longer align with the removed context,

rendering it unreasonable in practical use.

Based on these concerns, we propose FIM-SE, which stands for Fill-In-the-Middle with both Starting and Ending character constraints. Our method enhances the organizational framework of FIM (Bavarian et al., 2022) to concurrently address the two scenarios mentioned earlier. In simple terms, we transfer the random-span infilling task to the multi-line infilling task. Specifically, after random character level splitting, we utilize two distinct special tokens to mark the *Last line of the Prefix* (*L-Prefix*) and the *First line of the Suffix* (*F-Suffix*). The model is then tasked to generate text at line level that starts with *L-Prefix* and ends with *F-Suffix*. Their inclusion in the prompt simplifies the task for the model, facilitating the generation of text that seamlessly starts with *L-Prefix* and ends with *F-Suffix*. Overall, this method is designed to unlock the capabilities of decoder-only models in infilling tasks.

The core contribution of the paper is that we design a novel training method for the infilling task, a solution designed to effectively mitigate conflicts mentioned above in infilling tasks. Our method can effectively eliminate any potential inconsistencies and earnestly guarantee that the model’s output aligns cohesively with the given context. Extensive experiments demonstrate the effectiveness of the proposed method on infilling tasks while not compromising code generation capabilities. Based on Code Llama 13B, our approach not only achieves an 8.8% enhancement in the Humaneval random-span infilling task, with substantial improvements of 11.5% and 10.7% in the single-line and multi-line infilling tasks respectively, but also maintains minimal impact on the model’s performance in code generation tasks.

2 Related Work

2.1 Large Language Models for Infilling

Various Large Language Models (LLMs) have been developed for general generation tasks (Touvron et al., 2023a; Xiong et al., 2023; Chowdhery et al., 2023; Penedo et al., 2023; Jiang et al., 2023; Yang et al., 2023; Bai et al., 2023; Anthropic, 2024; Jiang et al., 2024; Anil et al., 2023a). Most of these models adopt a left-to-right autoregressive generation approach due to its effectiveness, as validated by research such as GPT (Radford et al., 2018, 2019; Brown et al., 2020; Ouyang et al., 2022; OpenAI, 2023). In the realm of code-related tasks, where

infilling is essential, LLMs are specifically trained for this task. For instance, InCoder (Fried et al., 2023) utilizes a causal masking objective, while SantaCoder (Allal et al., 2023), StarCoder (Li et al., 2023), and Code Llama (Rozière et al., 2023) employ the fill-in-the-middle technique introduced by FIM (Bavarian et al., 2022).

2.2 Text Infilling Models

The infilling task plays a crucial role in numerous real-world applications, including document editing¹ and code completion². Three common Transformer (Vaswani et al., 2017) architectures are capable of executing this task, *i.e.*, encoder-only, encoder-decoder, decoder-only. In encoder-only architectures, masked language modeling is employed as the pre-training task, exemplified by models like BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019). These models are designed to infill brief spans, ranging from a single token (Devlin et al., 2019) to a word (Cui et al., 2021), and even several contiguous tokens (Joshi et al., 2020). In encoder-decoder architectures, a common approach involves masking several tokens in the encoder and then tasking the model with decoding the complete sentence, as exemplified by MASS (Song et al., 2019). Additionally, models like BART (Lewis et al., 2020a) and T5 (Raffel et al., 2020) have introduced an infilling noising method. This technique replaces multiple tokens with a single mask token, challenging the model to decode the masked span. In decoder-only architectures, several methods are employed for infilling tasks. The Insertion Transformer (Stern et al., 2019) instructs the model to first determine the location for the next token, followed by the token prediction itself. Meanwhile, GLM (Du et al., 2022), CM3 (Aghajanyan et al., 2022), and InCoder (Fried et al., 2023) adopt a different approach. They shift the target span to the end of the context, employing left-to-right autoregressive modeling for training. In addition, MIM (Nguyen et al., 2023) proposed to use both forward and backward LMs that share parameters to significantly enhance the performance.

Most of these models are designed for token-level infilling tasks, which often don't align with real-world applications due to the incomplete nature of the final token in actual prompts. FIM (Bavarian et al., 2022) explored various levels of spans, *i.e.*, line level, token level, and character

level. As shown in their results, models trained with line-level or token-level spans perform poorly on character-level infilling tasks. To enhance the performance of models trained on token-level spans in character-level infilling tasks, token healing was proposed to fix tokenization artifacts that normally arise at the boundary between the end of a prompt and the beginning of a set of generated tokens³. While it effectively bridges the gap between the prefix and generated text, it falls short in handling the transition between generated text and the suffix, highlighting the need for further research in character-level infilling.

3 Preliminaries

In this section, we provide a straightforward introduction to the FIM (Bavarian et al., 2022) method and conduct a theoretical analysis of how inconsistent labeling affects the model's perplexity.

3.1 Fill-In-the-Middle (FIM)

FIM is designed to train models to complete the central sections of documents. This approach involves joint training on a combined dataset of traditional left-to-right sequences and data transformed by FIM, with an infilling rate reaching as high as 90%. According to experimental results, FIM maintains the autoregressive test losses of the left-to-right models without incurring significant costs, and it only slightly impacts the performance in downstream evaluations (Allal et al., 2023).

In a particular document, FIM segments a document into three distinct parts: the prefix, the middle, and the suffix. It introduces three levels of segmentation: single-line, multi-line, and random-span. Because random-span is more in line with actual usage conditions, previous studies (Rozière et al., 2023; Li et al., 2023) usually trained the model with random-span level splitting. After splitting, it moves the middle piece to the end as

$$\text{doc} \rightarrow (\text{pre}, \text{mid}, \text{suf}) \rightarrow (\text{pre}, \text{suf}, \text{mid}),$$

then concatenate the three pieces using special tokens as

$$\langle \text{PRE} \rangle \text{pre} \langle \text{SUF} \rangle \text{suf} \langle \text{MID} \rangle \text{mid} \langle \text{EOT} \rangle.$$

This mode is termed Prefix-Suffix-Middle (PSM) mode. Additionally, FIM introduced the Suffix-Prefix-Middle (SPM) mode, which interchanges

¹<https://copilot.microsoft.com>

²<https://github.com/features/copilot>

³https://github.com/guidance-ai/guidance/blob/main/notebooks/tutorials/token_healing.ipynb

the positions of the prefix and suffix. A variant of the SPM mode is also proposed, maintaining the same structure as the PSM mode. Detailed descriptions of these modes are provided in Appendix A.

3.2 Impact of Inconsistent Labels

When the FIM method employs the random-span approach, a training sample can contain up to four sub-tokens. This can potentially lead to inconsistent labels that indicate the same input but with different labels. This issue becomes particularly critical when the model is required to predict a sub-token. In Section 1, we construct a simple experiment to illustrate that this inconsistency can significantly affect the model’s perplexity. Here, we offer a theoretical analysis to further elucidate this phenomenon.

We are considering a classification task involving n classes, where each sample’s label is associated with one of m different categories across various training instances. Let $\mathbf{y} \in \{0, 1\}^n$ represent the actual label, and $\hat{\mathbf{y}} \in \mathbb{R}^n$ represent the predicted probabilities. In this context, the cross-entropy loss is computed as

$$\mathcal{L} = - \sum_{i=0}^n \mathbf{y}_i \log \hat{\mathbf{y}}_i. \quad (1)$$

Assuming that the first m elements of $\mathbf{y}$ (i.e., $\mathbf{y}_1, \dots, \mathbf{y}_m$) are set to 1, while the rest are 0, the loss function is defined as

$$\mathcal{L}(\hat{\mathbf{y}}) = - \sum_{i=0}^m \log \hat{\mathbf{y}}_i. \quad (2)$$

Then our objective can be expressed as

$$\hat{\mathbf{y}}^* = \arg \max \mathcal{L}(\hat{\mathbf{y}}), \quad s.t. \sum_{i=0}^n \hat{\mathbf{y}}_i = 1. \quad (3)$$

Here, we introduce the concept of the Lagrange Multiplier, which enables us to formulate the target function as

$$\mathcal{L}(\hat{\mathbf{y}}, \lambda) = - \sum_{i=0}^m \log \hat{\mathbf{y}}_i - \lambda \left(\sum_{i=0}^n \hat{\mathbf{y}}_i - 1 \right). \quad (4)$$

We then calculate the partial derivatives of $\hat{\mathbf{y}}_1, \dots, \hat{\mathbf{y}}_m$ respectively, which allows us to obtain

$$\frac{\partial \mathcal{L}(\hat{\mathbf{y}}, \lambda)}{\partial \hat{\mathbf{y}}_i} = -\frac{1}{\hat{\mathbf{y}}_i} - \lambda \quad \text{when } i = 1, \dots, m. \quad (5)$$

By setting these derivatives to zero, we obtain

$$\hat{\mathbf{y}}_1^* = \dots = \hat{\mathbf{y}}_m^* = -\frac{1}{\lambda}. \quad (6)$$

Since $\hat{\mathbf{y}}_{m+1}, \dots, \hat{\mathbf{y}}_n$ are not included in the objective function of Eq. (3), and given that the logarithm is a monotonically increasing function, setting these values to 0 would maximize the objective function. Consequently, the condition is formulated as $\sum_{i=1}^m \hat{\mathbf{y}}_i = 1$. By incorporating this condition, we derive

$$\hat{\mathbf{y}}_1^* = \dots = \hat{\mathbf{y}}_m^* = \frac{1}{m}. \quad (7)$$

We have now completed the proof, demonstrating that when a data point is labeled differently across various samples, the model tends to assign an equal probability of $\frac{1}{m}$ to each label. This behavior leads to a large perplexity of the model, which further suggests its limited modeling capability.

We consider this proof to be a microscopic explanation, which demonstrates that the perplexity of sub-tokens will be higher. In contrast, macroscopically speaking, we assume that the probability of the next token prediction obeys a certain distribution. Then, these sub-tokens are outliers. The presence of several outliers in each piece of training data will result in a low confidence in the model, that is, a high degree of perplexity.

This phenomenon is notably detrimental in sensitive tasks. For example, the initially predicted token in practical completion is usually a sub-token. The higher perplexity of the first token has little impact on the overall quality of the generated text, but in some sensitive tasks such as code completion, even a minor error in any token can result in program malfunction.

4 Methodology

In this section, we introduce the proposed method. We begin by outlining the training process with FIM-SE, followed by an explanation of the inference procedure. Finally, we delve into more training details and highlight the distinctions between our approach and the traditional FIM method.

4.1 FIM-SE Training

The core idea of FIM-SE is to ensure that the tokens predicted by the model are complete, thereby circumventing the issue of large perplexity associated with sub-tokens. Specifically, we shift from character-level to line-level random splitting in training data construction and then reconstruct the prompt to keep the ability of the model on the character-level infilling tasks.

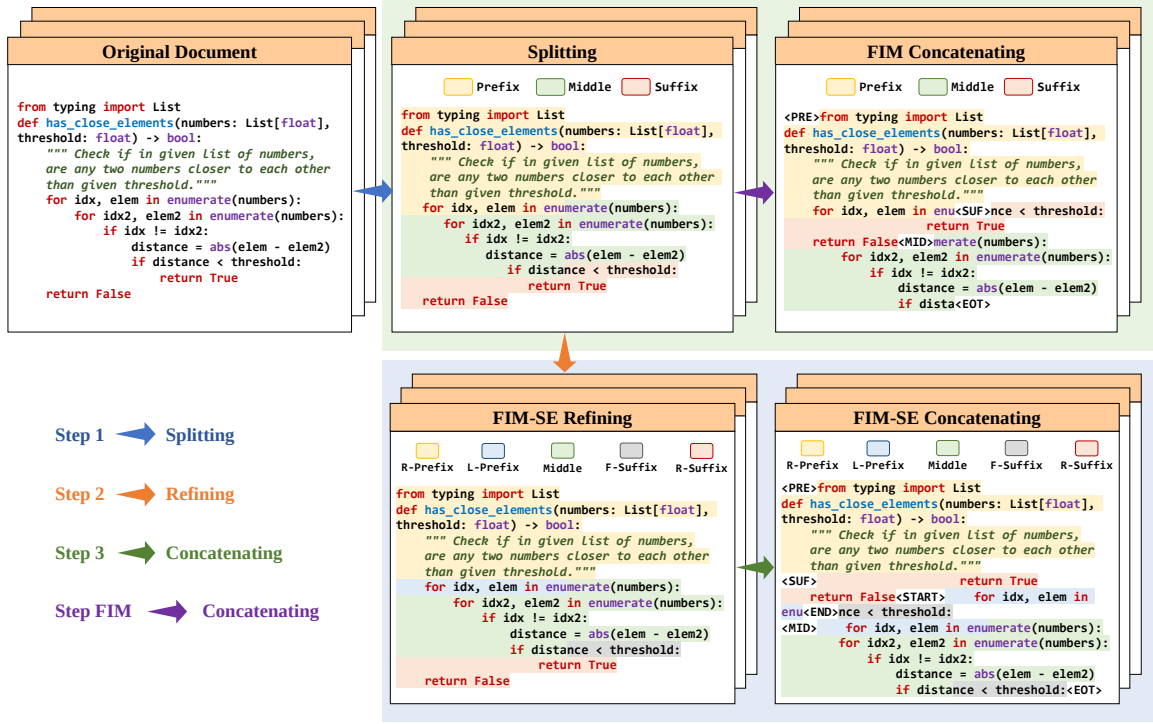


Figure 2: An overview of the difference between FIM and the proposed FIM-SE. Here, the green background indicates vanilla FIM and the blue background indicates our FIM-SE.

As shown in Figure 2, our process for forming the final training sample from a specific document involves three distinct steps. (1) Splitting: we split the original document into three pieces at the character level, namely the prefix, the middle, and the suffix (2) Refining: we distinguish between the last line of the prefix and the first line of the suffix, denoting them as *L-Prefix* and *F-Suffix*, respectively. Correspondingly, we label the remaining lines of the prefix and the suffix as *R-Prefix* and *R-Suffix*. (3) Concatenating: we concatenate all these sections in the following order along with their special tokens as

```
<PRE> R-Prefix <SUF> R-Suffix
<START> L-Prefix <END> F-Suffix
<MID> L-Prefix Middle F-Suffix <EOT>.
```

When tokenizing a sample, we tokenize each section individually and then concatenate them with the special tokens, which ensures that special tokens will not be cut or merged.

4.2 FIM-SE Inference

During the inference stage, the model can be employed for left-to-right generation in a standard manner. When working with an arbitrary location

within an existing document, we establish the preceding lines as *R-Prefix* and the following lines as *R-Suffix*. For the specific line at the target location, the text before this point is termed *L-Prefix*, and the text following it is named *F-Suffix*. Subsequently, a span is generated to be inserted at this location by autoregressively sampling tokens from the structured prompt

```
<PRE> R-Prefix <SUF> R-Suffix
<START> L-Prefix <END> F-Suffix<MID>.
```

This process continues until the “<EOT>” (End of Text) token is produced.

After obtaining the generation, we verify if it begins with the *L-Prefix* and ends with the *F-Suffix*. If the generation does not adhere to these criteria, we classify the infilling endeavor as unsuccessful. Conversely, if the criteria are met, we eliminate the *L-Prefix* from the beginning and the *F-Suffix* from the end, considering the remaining text as the completed segment.

4.3 Learning and Discussion

Training Details. We train our models using the StarCoder code corpus⁴, a carefully curated dataset

⁴<https://huggingface.co/datasets/bigcode/starcoderdata>

sourced from The Stack, encompassing 92 languages. To ensure consistency, we exclude categories such as GitHub issues, GitHub commits, and Jupyter Notebooks, which possess distinct column structures. Additionally, we remove flags marking repositories, files, and stars to maintain a focus on the pure code content in the remaining files. After gathering the data, we process it using the previously described method with a 90% FIM rate, following the methodologies of existing studies (Bavarian et al., 2022; Allal et al., 2023; Rozière et al., 2023). It’s important to note that we exclusively employ the PSM format depicted in Figure 2, as the SPM variant used in prior research (Bavarian et al., 2022) lacks a separator between the prefix and middle, potentially leading to model confusion. We conduct experiments and give an experimental analysis in Section 5.3.

Discussion. Compared to previous masked language modeling on encoder-only models and encoder-decoder models, our method excels in character-level infilling. While these traditional methods primarily concentrate on token-level infilling, this approach often falls short in numerous industry applications, as user text seldom forms complete tokens. Compared to vanilla FIM (Bavarian et al., 2022), our method also has the following two merits. Firstly, our method ensures that tokens following “<MID>” are complete, eliminating the need for sub-token predictions during inference and thereby mitigating the effects of the large perplexity of the model on sub-tokens. Secondly, our method transforms character-level infilling into line-level infilling. This unification of formats enhances transfer across different levels, significantly augmenting the efficacy of FIM training.

5 Experiments

In this section, we construct experiments to demonstrate the effectiveness of our method. Due to space limitations, we have constructed more experiments in Appendix B.

5.1 Experimental Setup

Datasets. Following FIM (Bavarian et al., 2022), we use code to test our methods. Because we can use test suites to evaluate the correctness of samples in our tasks even when evaluating long samples from open-ended generations. Specifically, we use three levels of infilling benchmarks, namely random-span, single-line, and multi-line.

All of them are constructed from Humaneval benchmarks (Chen et al., 2021). Since other infilling benchmarks such as Return Type Prediction and Docstring Generation focus on token-level infilling, we do not use these benchmarks.

Implementation Details. We continually pre-train four models with our methods, *i.e.*, StarCoder-1B, StarCoder-15B (Li et al., 2023), Code Llama 7B, and Code Llama 13B (Rozière et al., 2023). We employ AdamW (Loshchilov and Hutter, 2019) optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$ and weight decay of 0.1. Following previous study (Gupta et al., 2023), we set the peak learning rate to 3×10^{-5} and use a cosine schedule without warm-up. We use a batch size of 4M tokens which are presented as sequences of 8K tokens each for StarCoder and 16K tokens each for Code Llama. We train each model on 20B tokens in total. To efficiently train the computationally intensive models, we simultaneously employ DeepSpeed (Rajbhandari et al., 2020) and Flash Attention 2 (Dao, 2023). On 32 NVIDIA A800 80GB GPUs, StarCoder-1B, StarCoder-15B, Code Llama 7B, and Code Llama 13B take 14 hours, 140 hours, 75 hours, and 138 hours, respectively.

5.2 Results

Baselines. We compare FIM-SE with previous state-of-the-art methods, including InCoder (Fried et al., 2023), FIM (Bavarian et al., 2022), Codex (Chen et al., 2021), StarCoder (Li et al., 2023) and Code Llama (Rozière et al., 2023). For other code models such as CodeGeeX (Zheng et al., 2023) and OctoCoder (Muennighoff et al., 2023), we do not use them as baselines since they have not undergone infilling pre-training. Since we focus on character-level infilling, models focused on token-level infilling also do not be considered baselines. Because these models cannot effectively handle the character-level infilling task (Bavarian et al., 2022).

Random-span. As shown in Table 2, our proposed method demonstrates notable improvements in random-span infilling tasks across four models, specifically achieving gains of 4.7%, 1.3%, 8.1%, and 8.8%. Notably, the enhancement in StarCoder-15B is comparatively modest. This could be attributed to the fact that StarCoder has undergone pre-training with four epochs on a total of 1TB tokens, in contrast to Code Llama’s pre-training on 500B tokens, resulting in a more refined model fit. Comparing StarCoder-15B with StarCoder-1B, the

Model	Size	Training Methods	random-span	single-line	multi-line	Humaneval	MBPP
InCoder FIM code-davinci-002	6B	Causal Masking	-	69.0	38.6	15.0	19.4
	7B	FIM-SPM	55.1	75.1	44.1	-	-
	175B	-	74.2	91.6	69.9	44.5	55.4
StarCoder	1B	FIM-PSM FIM-SE-PSM	44.1* 48.8 (+4.7)	64.3* 72.6 (+8.3)	30.8* 37.1 (+6.3)	15.2 16.5	22.6* 25.6
	15B	FIM-PSM FIM-SE-PSM	66.4* 67.7 (+1.3)	83.8* 85.8 (+2.0)	53.7* 57.4 (+3.7)	30.4 30.5	43.2* 44.6
Code Llama	7B	FIM-SPM	39.0	83.3	50.8	33.5	41.4
		FIM-PSM	59.7	74.1	48.2		
		FIM-SE-PSM	67.8 (+8.1)	84.9 (+10.8)	57.2 (+9.0)	30.5	41.4
	13B	FIM-SPM	41.9	85.6	56.1	36.0	47.0
		FIM-PSM	63.6	75.9	51.0		
		FIM-SE-PSM	72.4 (+8.8)	87.4 (+11.5)	61.7 (+10.7)	37.2	50.2

Table 2: Pass@1 accuracy on Humaneval infilling datasets. Results evaluated on our end are marked with “*”, while those unavailable are left blank. Note that StarCoder was evaluated using a cleaned and smaller version of MBPP so we conducted a re-evaluation.

small model trained on the same tokens has more gain, suggesting that the consistent training approach of our method is particularly beneficial for smaller models in achieving better fit. Comparing StarCoder-15B with Code Llama 13B, the model with a similar size using fewer tokens achieves better results. This indicates that the consistent training approach of our method accelerates the fitting process in larger models.

Single-line and Multi-line. The proposed method demonstrates notable improvements in both single-line and multi-line infilling tasks. For instance, based on Code Llama 13B, our method surpasses FIM by 11.5% and 10.7% in single-line and multi-line infilling tasks, respectively. This enhancement can be attributed to two key factors. Firstly, our method integrates character-level and line-level processing, significantly enhancing the model’s line-level infilling capabilities. Secondly, it avoids the inclusion of any sub-tokens after the “<MID>” token, which sharpens the model’s accuracy in predicting the initial token. In contrast, in the standard FIM, the first token following “<MID>” is typically a sub-token during training, while the model is adopted to predict a complete token in the line-level infilling tasks during the inference stage. A comprehensive case study is provided in Appendix B.2 for further illustration.

Code Generation Task. We also report results on Humaneval (Chen et al., 2021) and MBPP (Austin et al., 2021). As shown in Table 2, our method has minimal impact on the model’s performance in the two code generation tasks (Note that we

Methods	FIM-SE	w/o LF-Loss
random-span	0.488	0.488
single-line	0.726	0.716
multi-line	0.371	0.369
Test loss	0.847	0.834

Table 3: Effect of training loss on sub-tokens. The metric is Pass@1 accuracy. Here, LF-Loss denotes the loss for tokens in *L-Prefix* and *F-Suffix*.

cannot reproduce the result of Code Llama 7B on Humaneval, just 29.9% in our environment). In summary, FIM-SE demonstrates a remarkable ability to improve infilling tasks without compromising code generation capabilities.

5.3 Detail Analysis

Impact of Inconsistent Labels. As mentioned in Section 1 and Section 3.2, we discussed how FIM leads to inconsistent labels during training at split points. This phenomenon results in large perplexity on sub-tokens, subsequently diminishing the model’s accuracy in generating sub-tokens. To investigate this effect, we conducted an experiment based on the StarCoder-1B. Specifically, we adjusted the temperature within the range of [0, 1.4] and compared the performance of models trained using both FIM-SE and FIM in generating 20 completions to estimate the Pass@1 rate. Figure 3 illustrates that the performance gap between the FIM-SE and FIM generators widens as the temperature increases, highlighting the larger perplexity associated with models trained using FIM.

Furthermore, we evaluate the impact of inconsis-

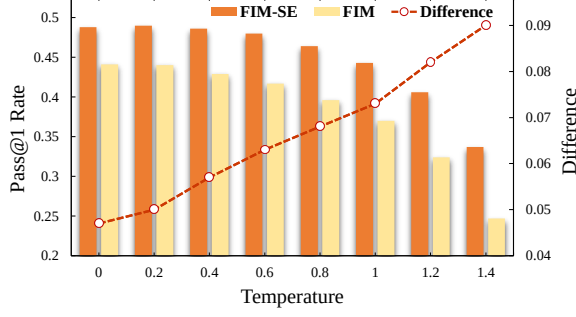


Figure 3: Performance on Humaneval random-span infilling task with different temperatures. The line denotes the difference between FIM-SE and FIM. Note that when the temperature surpasses 1.4, both models output noisy text and show very low performance.

Methods	random-span	single-line	multi-line
FIM-SE	0.488	0.726	0.371
SPM v1	0.492	0.703	0.374
SPM v2	0.013	0.085	0.088
SPM v3	0.090	0.717	0.383

Table 4: Comparison between different SPM format variants and FIM-SE. The metric is Pass@1 accuracy.

tent labels on training. Specifically, we mask the loss for tokens in *L-Prefix* and *F-Suffix*, ensuring that only complete tokens contribute to loss calculations. As shown in table 3, computing losses for *L-Prefix* and *F-Suffix* led to a slightly higher test loss without significantly affecting performance. This could be attributed to the minimal proportion of sub-tokens, as the presence of up to four sub-tokens per sample had a negligible impact on the final test results. In summary, while the loss of sub-tokens in training scarcely affects performance, the presence of sub-tokens in prediction objectives markedly influences performance.

Comparison with SPM Mode. In previous studies, the Suffix-Prefix-Middle variant had better performance in most cases (Bavarian et al., 2022; Rozière et al., 2023). Here, we explore how to combine our method with SPM mode based on StarCoder-1B. Specifically, we designed the following three prompt formats for SPM mode. We train the model using these formats and the PSM mode, equally distributed across 20 billion tokens.

- (1) SPM v1: “<SUF> *R-Suffix* <PRE> *R-Prefix* <START> *L-Prefix* <END> *F-Suffix* <MID>”, which add the constraints before the middle to the vanilla SPM mode.
- (2) SPM v2: “<PRE> <SUF> *R-Suffix* <START>

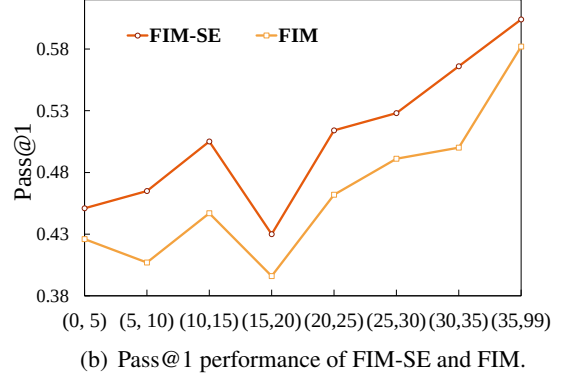
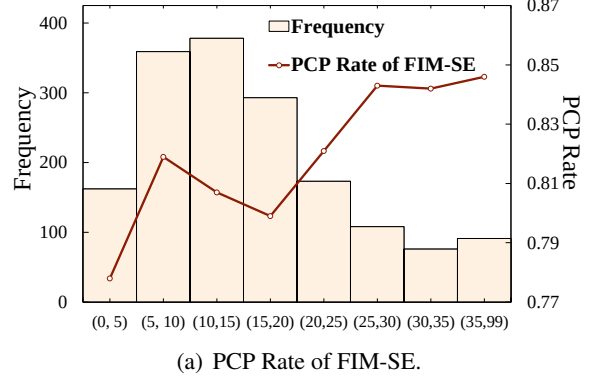


Figure 4: Statistics of length of *L-Prefix* and *F-Suffix*.

L-Prefix <END> *F-Suffix* <MID> *R-Prefix*”, which add the constraints before the middle to the variant SPM in FIM.

- (3) SPM v3: “<PRE> <SUF> *R-Suffix* <MID> *R-Prefix* <START> *L-Prefix* <END> *F-Suffix*”, which add the constraints after prefix to the variant SPM in FIM.

Table 4 presents all comparison results of the three variants. As we can see, SPM v2 and SPM v3 perform worse on random-span infilling tasks. This occurs because there is no separator between the prefix and the middle, leading to conflicts with the PSM mode, regardless of where the restriction is inserted. In contrast, SPM v1 and PSM perform almost the same because there is no conflict. To maintain consistency with the pre-trained models (Li et al., 2023; Rozière et al., 2023), we adopt the PSM mode.

Analysis of Post-Check during Inference. As we mentioned in Section 4.2, it’s essential to verify if the generation begins with the *L-Prefix* and ends with the *F-Suffix*. Here, we perform statistical analysis on the success rate of the model based on StarCoder-1B. We focus on the Post-Check Pass Rate (PCP Rate), which quantifies the percentage

of model outputs complying with the post-check criteria, *i.e.*, starting with the *L-Prefix* and ending with the *F-Suffix*. We then examine the correlation between the PCP Rate and the average length of the *L-Prefix* and *F-Suffix*. Additionally, we analyze the Pass@1 rates for FIM-SE and FIM across varying lengths of these prefixes and suffixes.

As shown in Figure 4, the PCP Rate increases with length, suggesting that longer *L-Prefixes* and *F-Suffixes* provide more guidance for the model’s text completion. Moreover, the Pass@1 metrics for both FIM-SE and FIM also support this, showing enhanced performance with extended *L-Prefixes* and *F-Suffixes*. Across all lengths, FIM-SE consistently outperforms the standard FIM, demonstrating the effectiveness of our approach.

6 Conclusion

In this paper, we showed that traditional infilling techniques struggle with managing the boundaries of prefixes and suffixes. To address this, we introduced a novel approach, referred to as FIM-SE. Our method transforms the random-span mode to multi-line mode by removing the *L-Prefix* and *F-Suffix*. We further incorporated two special tokens to delineate the two incomplete lines, thereby guiding the generation. Extensive experiments reveal that our approach surpasses existing baselines with a clear edge. In future work, we plan to explore the adaptation of our method to the variant SPM mode, which holds the promise of even better performance.

7 Limitations

The primary limitation of this study is the inability of the proposed method to accommodate the variant SPM mode, previously established as superior by prior research. This challenge arises due to the absence of a distinct delimiter between the prefix and middle, impeding our capacity to guide the model on the commencement point for completion and to appropriately position the prompt that instructs the model to start with *L-Prefix* and end with *F-Suffix*. In future endeavors, we plan to explore adapting our method to the variant SPM mode, to achieve better performance. Another limitation of this paper is the probability that our proposed method fails to complete tasks when the generation neither starts with the *L-Prefix* nor ends with the *F-Suffix*. For example, the fail rates of StarCoder-1B and StarCoder-15B are 18.7% and 9.4%, respectively. This issue is a primary factor

impacting model performance. Future work will concentrate on improving the post-check pass rate by developing more comprehensive prompts and refining constraint decoding.

8 Ethics Statement

In this paper, we utilized the StarCoder dataset (Li et al., 2023). This dataset has been made publicly available for academic purposes. The creators of the StarCoder dataset have transparently disclosed its derivation from The Stack v1.2 (Kocetkov et al., 2022). Importantly, The Stack v1.2 is compiled from a collection of GitHub repositories, all of which operate under permissive licenses. This ensures that the data’s utilization aligns with the original authors’ intentions and the legal frameworks governing open-source contributions. In conclusion, the application of the StarCoder dataset in our study complies with the ethical guidelines for research data usage, aligning with the broader principles of academic honesty and the responsible conduct of research.

Acknowledgment

This project is funded in part by National Key R&D Program of China Project 2022ZD0161100, by the Centre for Perceptual and Interactive Intelligence (CPII) Ltd under the Innovation and Technology Commission (ITC)’s InnoHK, by General Research Fund of Hong Kong RGC Project 14204021. Hongsheng Li is a PI of CPII under the InnoHK.

References

- Armen Aghajanyan, Bernie Huang, Candace Ross, Vladimir Karpukhin, Hu Xu, Naman Goyal, Dmytro Okhonko, Mandar Joshi, Gargi Ghosh, Mike Lewis, and Luke Zettlemoyer. 2022. [CM3: A causal masked multimodal model of the internet](#). *CoRR*, abs/2201.07520.
- Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Muñoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, Logesh Kumar Umapathi, Carolyn Jane Anderson, Yangtian Zi, Joel Lamy-Poirier, Hailey Schoelkopf, Sergey Troshin, Dmitry Abulkhanov, Manuel Romero, Michael Lappert, Francesco De Toni, Bernardo García del Río, Qian Liu, Shamik Bose, Urvashi Bhattacharyya, Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco Zocca, Sourab Mangrulkar, David Lansky, Huu Nguyen, Danish Contractor, Luis Villa, Jia Li, Dzmitry Bahdanau, Yacine Jernite, Sean Hughes, Daniel Fried, Arjun Guha, Harm de Vries, and Leandro von Werra. 2023.